엔터프라이즈 지식그래프 및 시스템 온톨로지 구축 아키텍처 보고서: 줌줌투어 메타데이터 자산화 전략

핵심 결론 요약

1. 줌줌투어의 1차 목표인 시스템 연동 추적, 600여 개 테이블 메타데이터 관리, 부서별 업무 프로세스 매핑을 달성하기 위한 최적 스택은 속성 그래프(LPG) 기반의 Neo4j Community Edition과 자동화 스키마 추출 도구 tbls의 결합으로 도출된다[1].
2. 단일 사내 서버(8 vCPU/32GB) 자원 제약과 비용 0원 제약 조건을 충족하기 위해, 마이크로서비스 구동 부하가 큰 엔터프라이즈 카탈로그 대신 경량 CLI 기반 스키마 추출 파이프라인을 구축한다[3].
3. Neo4j 커뮤니티 에디션의 주요 제약인 세부 권한 제어(RBAC) 부재는 AI 에이전트 연동 계층(MCP Server)의 읽기 전용 샌드박싱과 GitOps 단방향 승인 파이프라인을 통해 보완한다[4].
4. 여행 도메인(상품, 예약, 결제, 정산, 공급사)과 기술 메타데이터를 W3C ORG, PROV-O, schema.org 등 국제 표준 어휘 구조와 정렬하여 향후 데이터 상호운용성과 확장성을 확보한다.
5. 단일 개발 인력(주 1~2일 투입) 환경을 감안하여 예약-발권 핵심 비즈니스 프로세스 1건을 대상으로 4주 파일럿을 우선 수행하고, 20대 역량 질문 검증을 통과한 후 전사 시스템으로 단계적 확장을 전개한다.

1. 개념 체계: 온톨로지, 지식그래프, 속성 그래프, RDF/OWL의 상호 관계와 기술 선정

개념 정의 및 상호 관계

지식 관리 및 데이터 공학 분야의 핵심 용어들은 데이터의 구조화 수준과 논리적 엄밀성에 따라 층위가 나뉜다. 비전문가의 이해를 돕기 위해 각 개념의 기술적 본질을 비유를 통해 설명할 수 있다.

온톨로지(Ontology)는 특정 도메인 내에 존재하는 모든 개념(Concept), 개념 간의 관계(Relation), 그리고 그 도메인을 규정하는 규칙(Rule)을 컴퓨터가 이해할 수 있는 언어로 형식화한 개념적 설계도이자 표준 법전이다. 데이터베이스 설계로 치면 실제 레코드가 없는 논리적 ERD 또는 스키마 명세서에 해당한다.

지식그래프(Knowledge Graph)는 온톨로지라는 설계도를 바탕으로 현실 세계의 구체적인 개체(인스턴스)들과 그들 사이의 연결 관계를 거미줄 형태로 채워 넣은 실체화된 지식 네트워크이다. 시스템 명칭, 테이블 이름, 담당 부서, 실제 업무 흐름 등이 노드와 엣지로 결합된 전체 데이터 망을 의미한다.

속성 그래프(Labeled Property Graph, LPG)는 지식그래프를 컴퓨터 시스템에 저장하기 위해 설계된 실용적인 데이터 모델이다. 점(노드)과 선(관계) 자체에 키-값(Key-Value) 형태의 속성 데이터를 자유롭게 부여할 수 있는 레고 블록 구조를 띤다. 복잡한 시스템 아키텍처와 엔지니어링 메타데이터를 직관적으로 다룰 수 있어 소프트웨어 엔지니어링 실무에서 널리

채택된다.
RDF(Resource Description Framework)와 OWL(Web Ontology Language)은 W3C(월드와이드웹 컨소시엄)에서 제정한 국제 표준 데이터 표현 규격이다. RDF는 세상의 모든 지식을 주어(Subject), 술어(Predicate), 목적어(Object)라는 3단어 문장(트리플, Triple)으로 쪼개어 기술하는 엄격한 문법 체계이다. OWL은 여기에 기호 논리학 기반의 공리(Axiom) 체계를 더해, 컴퓨터가 사전 정의된 규칙을 바탕으로 새로운 사실을 스스로 유추할 수 있도록 만든 자동 추론(Reasoning) 프레임워크이다.

구분	개념적 비유	데이터 최소 표현 단위	다루는 핵심 대상	주요 기술 표준 및 구현체
온톨로지 **(Ontology)**	건축 설계 도면, 법률 및 표준 용어집	클래스(Class), 공리(Axiom)	비즈니스 개념, 업무 규칙, 도메인 정의	OWL, RDFS
지식그래프 **(KG)**	설계도대로 완공된 도시 인프라망	엔티티(Entity), 관계(Relation)	개념과 인스턴스가 통합된 지식 네트워크	Neo4j, GraphDB, Virtuoso[5]
속성 그래프 **(LPG)**	내부에 부가 정보를 기록하는 레고 블록	노드, 관계, 속성(Key-Value)	복잡한 관계 경로 추적, 시스템 연동도	Neo4j, Memgraph, Cypher[1]
RDF / OWL	모든 문장이 3단어로 구성된 조약문	주어-술어-목적어 (트리플)	전역 식별자(URI), 논리적 공리 기반 추론	Apache Jena, SPARQL, Protégé[8]

줌줌투어 목적에 부합하는 데이터 모델 판정

줌줌투어의 1차 목표는 여행 상품의 철학적·논리적 자동 추론 체계를 구축하는 것이 아니라, 서비스-모듈 간 연동 추적, 600여 개 테이블 및 컬럼의 구조 파악, 부서 업무 단계별 데이터 CRUD 매핑, AI 에이전트를 통한 영향도 분석이라는 매우 구체적인 엔지니어링 문제를 해결하는 데 있다.
이러한 목적에는 속성 그래프(LPG) 모델이 적합하다. RDF 모델을 적용할 경우, 테이블 노드 하나에 컬럼 타입, 한글 설명, 담당자, 생성일자 등의 속성을 부여할 때마다 독립된 트리플(선)이 4~5개씩 방사형으로 늘어나 그래프가 과도하게 비대해진다. 반면 LPG는 노드 내부에 속성을 캡슐화하여 보관할 수 있어 직관적이며 구조 단순성을 유지한다.
또한 LPG의 대표 질의 언어인 Cypher는 아스키 아트 형태((node)-[:RELATION]->(target))를 띠고 있어 관계형 데이터베이스와 SQL에 익숙한 사내 개발자가 단기간에 습득할 수 있다. 반면

RDF 질의 언어인 SPARQL은 문법적 복잡성으로 인해 비전문가와 개발자 모두에게 높은 진입 장벽을 유발한다[9].
AI 에이전트(LLM) 통합 측면에서도 최신 LLM(Claude, GPT 계열)은 오픈 웹에 축적된 코드를 바탕으로 자연어를 Cypher로 변환하는 능력(Text-to-Cypher)이 SPARQL 변환 능력보다 월등히 우수하다. 따라서 본 아키텍처에서는 개념 설계 단계에서 W3C 표준 온톨로지 구조를 수용하되, 물리적 저장 및 질의 계층은 속성 그래프(LPG)로 구현하는 실용적 접근법을 채택한다.

2. 오픈소스 및 무료 셀프호스팅 솔루션 기술 조사

단일 사내 서버(8 vCPU, 32GB RAM, Ubuntu 환경)에서 라이선스 비용 0원으로 운영 가능한 주요 오픈소스 및 무료 솔루션의 2025~2026년 기준 현황을 조사하였다.

Neo4j Community Edition

Neo4j Community Edition은 글로벌 시장에서 가장 널리 사용되는 속성 그래프 데이터베이스(LPG)이다[1]. 2025년과 2026년에 걸쳐 캘린더 버저닝(CalVer) 체계와 장기 지원 버전인 5.26 LTS를 병행 배포하고 있다[1].

- 라이선스: GPL v3 (GNU General Public License v3)가 적용된다[12]. Neo4j 공식 라이선스 명세에 명시된 바와 같이, 사내 내부 업무용(Internal Commercial Use) 셀프호스팅은 라이선스 비용 없이 합법적으로 사용할 수 있다[13]. Neo4j 소스코드를 수정한 후 이를 외부에 SaaS 형태로 서비스하지 않는 한 소스코드 공개 의무는 발생하지 않는다.
- 무료판 제약 사항: 상용 Enterprise 버전과 비교할 때 엄격한 기능 차등이 존재한다. 첫째, 역할 기반 접근 제어(RBAC)가 지원되지 않아 세부적인 읽기/쓰기 권한 분리가 불가능하며, 생성된 사용자는 전체 데이터베이스 권한을 갖는다[1]. 둘째, 온라인 라이브 백업 기능이 차단되어 있어 데이터 덤프(neo4j-admin database dump)를 수행하려면 데이터베이스를 오프라인으로 전환해야 한다[14]. 셋째, 단일 인스턴스(Single Instance) 구동만 지원하며 고가용성 클러스터링 및 멀티 데이터베이스 동시 활성화가 제한된다[1].
- 시스템 요구사항: 최소 4GB RAM, 권장 8GB~16GB RAM 환경을 요구하며 공식 Docker 이미지(neo4j:community)를 통해 간편하게 배포할 수 있다[18].

Memgraph

Memgraph는 실시간 분석과 저지연 데이터 처리를 목적으로 C++로 설계된 고성능 인메모리 속성 그래프 데이터베이스이다[7]. Cypher 질의 언어의 오픈 표준인 openCypher를 완전하게 지원한다[7].

- 라이선스: Business Source License (BSL 1.1)를 따르며, 핵심 알고리즘 라이브러리는 Apache 2.0으로 공개되어 있다[17]. Memgraph 공식 GitHub 저장소에 따르면, 상용 클라우드 서비스(DBaaS)로 재판매하지 않는 일반 기업의 내부 상업적 구축은 완전 무료로 허용된다[17].
- 무료판 제약 사항: 고가용성 복제 클러스터 구성 및 엔터프라이즈 모니터링 감사 기능에 일부 제약이 있다[17]. 인메모리 아키텍처 특성상 모든 그래프 데이터와 인덱스가 RAM에 상주해야 하므로, 데이터셋 규모가 가용 메모리를 초과하면 스왑이 발생하여 성능이 저하될 수 있다[17]. 향후 전체 엔터프라이즈 모듈의 완전한 Apache 2.0 라이선스 전환 여부는 배포 버전별 확인이 필요하다[20].
- 시스템 요구사항: C++ 기반으로 최소 2GB, 권장 8GB RAM 수준에서 원활하게 구동되며 공식 Docker 이미지(memgraph/memgraph-platform)를 제공한다[7].

Apache Jena Fuseki

Apache Jena Fuseki는 W3C 표준 시맨틱 웹 기술 스택을 완벽하게 구현한 순수 오픈소스 RDF 트리플 저장소 및 SPARQL 서버이다[9]. 2026년 기준 6.x 버전(최신 안정판 6.2.0)이 배포되고 있으며 Java 21 런타임을 필수로 요구한다[8].

- 라이선스: Apache License 2.0으로 배포되어 상업적 목적의 사내 구축에 아무런 법적 제약이 없다[8]. Apache Jena 공식 문서를 통해 확인 가능하다[24].
- 무료판 제약 사항: 단일 오픈소스 프로젝트이므로 유료 기능 차등은 없다. 그러나 TDB2 스토리지 엔진 기반으로 대규모 병렬 쓰기 트랜잭션이 발생할 때 락 경합이 발생하며[9], Docker Hub에 최신 버전 공식 이미지가 지속적으로 유지관리되지 않아 커뮤니티 빌드 이미지(stain/jena-fuseki, conceptkernel/jena-fuseki)를 참조하거나 직접 빌드해야 하는 운영 번거로움이 존재한다[8].
- 시스템 요구사항: 최소 4GB RAM, 권장 8GB RAM이며 Java 21 이상 환경에서 구동된다[8].

Ontotext GraphDB Free

Ontotext GraphDB는 온톨로지 추론 성능과 엔터프라이즈 RDF 처리에 특화된 그래프 데이터베이스이다[5].

- 라이선스: 독점 무료 라이선스(Proprietary Free) 체계로 운영되며, Ontotext GraphDB 공식 포털에서 라이선스 키를 요청하여 발급받아야 설치가 가능하다[5].
- 무료판 제약 사항: 무료 버전에는 실무 도입을 가로막는 치명적인 기술 제약이 존재한다. 시스템 내부적으로 최대 동시 쿼리 수가 2개로 강제 제한(Two Concurrent Queries Limit)되어 있어, 다수의 팀원이 AI 에이전트로 동시 질의를 수행할 경우 대기 큐 병목과 타임아웃이 발생한다[5]. 또한 생성 가능한 리포지토리 수가 최대 5개로 한정되며 엔터프라이즈 보안 및 클러스터링이 제외된다[6].
- 시스템 요구사항: 최소 8GB RAM, 권장 16GB RAM으로 자원 소모량이 크다[6].

tbls

tbls는 Go 언어로 작성된 데이터베이스 스키마 자동 추출, 문서화 및 린트(Lint) 도구이다[2].

- 라이선스: MIT License가 적용된 순수 오픈소스이다[2]. tbls 공식 GitHub에서 소스코드가 관리된다[2].
- 무료판 제약 사항: 단일 바이너리 CLI 도구이므로 인위적인 기능 제약은 없다[2]. 다만 독립적인 영속 저장소나 상시 웹 UI를 제공하지 않으며, 데이터베이스에 연결하여 스키마를 정형 JSON(schema.json) 또는 마크다운 파일로 역공학 출력하는 단방향 파이프라인 기능만 수행한다[2].
- 시스템 요구사항: 단일 Go 실행 파일로 수십 MB 수준의 메모리만 점유하며, 경량 Docker 컨테이너(ghcr.io/k1low/tbls)로 즉각 실행 가능하다[2].

OpenMetadata

OpenMetadata는 분산 환경의 메타데이터 수집, 계통 추적(Lineage), 데이터 품질 관리를 지원하는 통합 오픈소스 데이터 카탈로그이다[3].

- 라이선스: Apache License 2.0으로 배포된다[3]. OpenMetadata 공식 사이트에서 아키텍처를 확인할 수 있다[3].
- 무료판 제약 사항: 코어 자체의 인위적인 제약은 적으나, 플랫폼 구동을 위해 OpenMetadata 서버, OpenSearch(또는 Elasticsearch), 관계형 DB(MySQL/PostgreSQL),

인제스천 파이프라인 러너 등 최소 4~6개의 독립 컨테이너가 동시에 동작해야 한다[3].

- 시스템 요구사항: 최소 4 vCPU, 6GB~8GB RAM을 상시 점유하므로[3], 8 vCPU/32GB 단일 서버에서 그래프 데이터베이스 및 AI 서비스와 공존 구동할 경우 리소스 경합을 초래한다.

WebProtégé

WebProtégé는 미국 스탠퍼드 대학교 BMIR 연구팀이 개발한 웹 기반 협업 OWL 2 온톨로지 에디터이다[10].

- 라이선스: BSD 2-Clause License로 배포된다[29]. WebProtégé 배포 저장소를 통해 설치 파일이 제공된다[29].
- 무료판 제약 사항: 온톨로지 파일(OWL, RDF/XML, Turtle)의 생성 및 시각적 협업 편집 도구일 뿐, 실시간 Cypher나 SPARQL 질의를 처리하는 고성능 데이터베이스 역할을 수행할 수 없다[10]. 최신 5.x 계열 배포판은 Keycloak, MongoDB, GWT 서비스가 결합되어 있어 초기 설치 및 도커 구성의 복잡도가 높다[10].
- 시스템 요구사항: 최소 4GB RAM, 권장 8GB RAM 환경을 요구한다[10].

3. 솔루션 다차원 비교 분석

사내 인프라 조건(8 vCPU, 32GB RAM, 1명의 개발자가 주 1~2일 투입)과 팀원 협업 환경을 기준으로 7개 솔루션의 장단점을 다차원으로 평가한다.

비교 항목	**Neo4j Community**	**Memgraph**	**Apache Jena Fuseki**	**Ontotext GraphDB Free**	**tbls**	**OpenMetadata**	**WebProtégé**
데이터 모델	속성 그래프 (LPG)[1]	속성 그래프 (LPG)[7]	RDF 트리플[9]	RDF 트리플 / OWL[5]	관계형 스키마 메타[2]	하이퍼 그래프 카탈로그	OWL 2 온톨로지[10]
질의 언어	Cypher[1]	openCypher[7]	SPARQL 1.1[9]	SPARQL 1.1 / GraphQL[6]	CLI / 정적 JSON 파싱[2]	REST API / UI 검색	GUI 마우스 기반[10]
라이선스	GPL v3[12]	BSL 1.1 / Apache 2.0[17]	Apache 2.0[8]	독점 무료 (등록제)[6]	MIT[2]	Apache 2.0[3]	BSD 2-Clause[29]
사내 상업적	허용 (내부망	허용 (DBaaS	완전 허용[8]	허용 (라이선	완전 허용[2]	완전 허용[3]	완전 허용[28]

이용	구동)[13]	판매 외)[17]		스 발급)[6]			
무료판 핵심 제약	RBAC 없음, 오프라인 덤프만 지원, 단일 DB[1]	고가용 클러스터 및 모니터링 제약[17]	인위적 제약 없음, 대규모 쓰기 락 경합[9]	동시 쿼리 **2**개 제한, 최대 5개 리포지토리[5]	저장소 기능 부재 (단순 CLI 생성기)[2]	4~6개 컨테이너 구동으로 리소스 소모 큼[3]	편집 전용 (실시간 DB 쿼리 서비스 불가)[10]
비전문가 학습 난이도	보통 (직관적 Cypher)	보통 (Cypher 호환)[7]	높음 (SPARQL 학습)[9]	매우 높음 (시맨틱 웹)	매우 쉬움 (MD 문서)[2]	쉬움 (웹 UI 우수)[3]	보통 (GUI 에디터)[10]
시각화 및 웹 **UI**	Neo4j Browser 기본 제공[11]	Memgraph Lab 기본 제공[17]	Fuseki UI (기본 수준)[9]	GraphDB Workbench[5]	없음 (HTML/MD 렌더링)[2]	최상 (포털형 UI)[3]	우수 (트리 및 그래프 뷰)[28]
다중 협업 및 권한	다중 계정 가능, 역할 분리 불가[4]	CE는 기본 인증 중심[17]	Shiro Basic 인증[9]	기본 계정 관리 지원	N/A (Git PR 기반)[2]	완벽한 RBAC 및 워크플로우[3]	프로젝트별 공유 및 댓글[28]
AI / MCP 서버 연동	공식/Labs 수준 높은 MCP 제공 (mcp-neo4j)[32]	openCypher 기반 Neo4j 드라이버 호환[7]	커뮤니티 수준 SPARQL 래퍼 (불안정)	커뮤니티 SPARQL 래퍼 수준	로컬 파일시스템 MCP로 우회[34]	REST API 기반 Tool 호출	직접 연동 불가
MCP 읽기 전용	환경변수/툴 레벨 지원	프록시 설정 필요	엔드포인트별 수동 분리[9]	엔드포인트 분리	정적 파일 기반 읽기	API 토큰 권한	미지원

지원	(read_only)[4]			가능	전용	제어	
DB 스키마 자동 수집	미지원 (스크립트 개발 필요)	미지원 (일부 툴킷 존재)[20]	미지원 (외부 매퍼 필요)	Ontorefine 등 복잡한 수동 매핑	완벽 지원 **(MySQL, PG** 등 **14**개 **DB)**[2]	완벽 지원 **(**내장 커넥터 다수**)**[3]	미지원
설치·운영 난이도	매우 쉬움 (단일 컨테이너)[19]	매우 쉬움 (단일 컨테이너)	보통 (Java 컨테이너)[9]	쉬움 (단일 컨테이너)	매우 쉬움 (경량 Docker/CLI)[2]	매우 어려움 (복합 마이크로서비스)[3]	보통 (Compose 제공)[10]
권장 하드웨어 사양	2~4 vCPU / 4~8GB	2 vCPU / 4~8GB	2 vCPU / 4GB[8]	4 vCPU / 8~16GB	미미함 (수십 MB 수준)	4 vCPU / 8GB 이상[3]	2 vCPU / 4GB[10]
백업 및 복구 체계	오프라인 덤프 (neo4j-admin dump)[14]	스냅샷 + WAL 복제[17]	디렉토리 복제 / TDB2 덤프[9]	UI 내 덤프 및 익스포트	Git 저장소로 영구 버전 관리[2]	PostgreSQL 메타 백업[3]	MongoDB 덤프[10]
생태계 및 한국어 자료	전 세계 1위, 한국어 자료 방대[13]	성장세, 국내 레퍼런스 적음	학술 중심, 국내 자료 희소	상용 중심, 국내 자료 희소	글로벌 오픈소스 우수[2]	대형 글로벌 커뮤니티[3]	학술·의료 중심[28]

4. 권장 아키텍처 구성 및 무료판 제약 극복 방안

1순위 추천 조합**:** 실용적 자동화 파이프라인

단 1명의 개발자가 주 1~2일만 투입 가능한 자원 제약 하에서, 줌줌투어의 최적 조합은 tbls(스키마 추출기) + Neo4j Community Edition(지식그래프 저장소) + neo4j-contrib/mcp-neo4j-cypher(AI 인터페이스) + Git/GitHub Actions(버전 관리 및 거버넌스)의 결합이다[2].

전체 시스템 구성 흐름은 다음과 같다.

- 원천 **DB** 메타데이터 추출 계층: 운영 중인 MySQL 4대와 PostgreSQL 1대의 읽기 전용 계정 정보를 바탕으로, 일일 Cron 작업 또는 Docker 컨테이너를 통해 tbls를 실행한다[2]. tbls는 시스템 카탈로그를 분석하여 테이블, 컬럼, 외래키, 주석을 포함하는 정규화된 schema.json을 도출한다[2].
- 지식 저장 및 적재 파이프라인: 도출된 schema.json과 업무 프로세스 정의 문서는 파이썬 기반 변환 스크립트를 거쳐 Cypher MERGE 문으로 컴파일된다[27]. 이 데이터는 로컬 사내 서버에서 실행되는 Neo4j 5.26 LTS 또는 2026 Community Edition 인스턴스로 일괄 적재된다[1].
- **AI** 에이전트 인터페이스 계층: 사내 서버 내부망에 mcp-neo4j-cypher 컨테이너를 구동하고 Bolt 프로토콜을 통해 Neo4j와 연동한다[32]. 사내 15명의 사용자는 각자의 Claude Desktop, Claude Code, 또는 Cursor 에이전트에 MCP 프로토콜로 연결하여 자연어로 사내 지식을 탐색한다[32].

선정 및 탈락 사유

이 조합을 선정한 핵심 이유는 자원 효율성과 구축 속도에 있다. 개발자가 600여 개 테이블의 메타데이터 파서를 직접 작성하려면 수주의 공수가 소요되지만, tbls를 활용하면 불과 수 분 만에 정형 JSON 데이터를 확보할 수 있다[2]. 또한 Neo4j의 방대한 글로벌 커뮤니티와 공식에 준하는 Neo4j Labs의 MCP 서버 지원은 개발자가 겪을 수 있는 기술적 시행착오를 최소화한다[13]. Neo4j CE의 메모리 점유율(약 4GB~6GB)은 32GB RAM 서버 환경에서 안정적으로 공존할 수 있는 최적의 크기이다.

반면 Ontotext GraphDB Free는 동시 쿼리 수가 2개로 제한되어 있어 15명의 동시 AI 에이전트 요청을 처리할 수 없으므로 탈락하였다[5]. Apache Jena Fuseki는 다대다 업무 프로세스(BPMN) 경로 탐색 질의를 SPARQL로 작성할 때의 비효율성과 시각화 도구의 부재로 인해 탈락하였다[9]. OpenMetadata는 6개 이상의 컨테이너와 8GB 이상의 메모리를 상시 점유하며, 프로세스 단계별 테이블 CRUD 매핑과 같은 유연한 비즈니스 지식그래프를 구성하기에는 카탈로그 데이터 모델이 지나치게 경직되어 있어 배제되었다[3].

대안 아키텍처

- 대안 **1 (**고성능 인메모리 파이프라인**)**: tbls + Memgraph + Custom openCypher MCP
  - C++ 인메모리 기반 Memgraph를 활용하여 1ms 미만의 극도로 빠른 응답 속도를 확보하는 구성이다[7]. BSL 라이선스의 비즈니스 안전성을 확보할 수 있으나[17], Neo4j 생태계 대비 MCP 툴체인의 장기적 호환성을 지속 검증해야 한다.
- 대안 **2 (**카탈로그**-**그래프 분리형 파이프라인**)**: OpenMetadata (순수 DB 카탈로그) + Neo4j CE (비즈니스 프로세스 전용)
  - DB 스키마 관리는 OpenMetadata 웹 UI에 위임하고[3], 시스템 연동도와 업무 프로세스 관계만 Neo4j로 분리 구축한다. 조직 규모가 100명 이상으로 확대되고 전담 데이터 엔지니어링 팀이 신설될 경우 전환할 가치가 있다.

무료판의 약점 극복 운영 방안

Neo4j Community Edition의 결정적 약점인 역할 기반 접근 제어(RBAC) 부재와 온라인 백업 차단은 다음과 같은 엔지니어링 및 거버넌스 방식으로 보완한다[1].

첫째, 권한 격리를 네트워크 및 프로토콜 레벨에서 강제한다. Neo4j DB 포트(7474, 7687)는 외부 IP로 개방하지 않고 로컬 루프백(127.0.0.1)에만 바인딩한다[11]. 팀원들은 DB에 직접 접근하지 않고

중간의 mcp-neo4j-cypher 서버를 경유한다[32]. MCP 서버 실행 환경변수에 read_only=true를 강제 적용하여 에이전트가 CREATE, SET, DELETE, MERGE 등의 수정 쿼리를 일절 전송하지 못하도록 원천 차단한다[4]. 과거 버전(< 0.6.0)에서 보고된 APOC 우회 쓰기 취약점(CVE-2026-35402)을 방지하기 위해 패치 완료된 최신 v0.6.0 이상의 컨테이너 이미지를 고정 배포한다[33].

둘째, 데이터 쓰기는 오직 Git 저장소의 PR(Pull Request) 병합 파이프라인(GitOps)을 통해서만 수행되는 단방향 파이프라인을 구축한다. 팀원들의 지식 변경 제안은 Git에서 승인된 후, 배포 러너가 로컬 관리자 권한으로 Neo4j에 배치 반영한다.

셋째, 온라인 백업 부재는 트래픽이 없는 심야 시간대의 컨테이너 오프라인 덤프 스케줄링으로 해결한다. 매일 새벽 04:00 CronJob을 통해 10초간 Neo4j 컨테이너를 중지(docker stop)하고, neo4j-admin database dump를 수행한 후 즉시 컨테이너를 복구(docker start)한다[14]. 생성된 .dump 파일은 사내 보조 스토리지로 암호화 동기화한다.

5. 줌줌투어 도메인 온톨로지 설계 명세

클래스 및 관계 목록

여행 플랫폼의 비즈니스 도메인과 물리적 IT 자산, 조직 업무를 상호 연계하기 위해 17개의 핵심 클래스(라벨)를 정의한다.

도메인 분류	클래스**(Node Label)**	설명
비즈니스 엔티티	TourProduct	줌줌투어가 판매하는 개별 여행, 투어, 티켓, 액티비티 상품
	Booking	고객이 상품을 예약하여 생성된 비즈니스 거래 건
	Payment	외부 PG사를 통해 처리된 결제 및 환불 트랜잭션
	CustomerMeta	고객 식별 체계 및 회원 등급 (개인 식별 실데이터 제외)
	Partner	현지 투어 및 상품을 공급하고 정산을 수취하는 파트너사/가이드
	Settlement	파트너에게 지급되는 판매 대금 정산 및 송금 내역

	CSTicket	고객의 문의, 일정 변경, 클레임 등 고객 지원 접수 건
	Voucher	예약 확정 후 파트너 API 연동 또는 수동 발급되는 여행 이용권
기술/인프라 메타	System	예약 API, 파트너 어드민, CMS, 정산 배치 등 개별 소프트웨어
	ExternalAPI	외부 결제대행사(토스, 이니시스), 항공사, 현지 공급사 API
	Database	물리적/논리적 DB 인스턴스 (MySQL 1~4, PostgreSQL 1개)
	Table	데이터베이스 내 개별 물리 테이블 (전사 600여 개 대상)
	Column	테이블을 구성하는 개별 컬럼 메타데이터 및 속성 정의
조직/프로세스	Department	사내 공식 부서 (상품기획, 운영, CS, 정산, 마케팅, 개발)
	Role	부서 내에서 정의된 실무 직무 및 권한 역할
	BusinessProcess	단위 비즈니스 워크플로우 (예: 투어 예약 확정 및 바우처 발권)
	ProcessStep	워크플로우를 구성하는

		개별 실행 단위 단계

시작 노드 라벨	관계 타입**(Edge Type)**	도착 노드 라벨	관계 설명
System	CALLS_API	ExternalAPI	시스템 모듈이 외부 PG 및 공급사 API를 호출하는 연동 관계
System	USES_DB	Database	시스템이 데이터를 저장하고 조회하기 위해 접근하는 DB 인스턴스
Database	HAS_TABLE	Table	데이터베이스 인스턴스에 소속된 테이블 소유 구조
Table	HAS_COLUMN	Column	테이블을 구성하는 컬럼 필드 관계
Table	FK_REFERENCES	Table	물리적 또는 논리적 외래키(Foreign Key) 참조 관계
Table	REPRESENTS_ENTITY	비즈니스 엔티티	물리 테이블이 어떤 비즈니스 개념 데이터를 영속화하는지 매핑
Department	HAS_ROLE	Role	부서 조직에 배속된 실무 직무 역할
Department	OWNS_SYSTEM	System	시스템의 비즈니스 오너십 또는 기술 유지보수 책임 부서
Role	RESPONSIBLE_FOR	BusinessProcess	특정 업무

			워크플로우를 주관하고 완수할 책임이 있는 주체
BusinessProcess	COMPOSED_OF	ProcessStep	프로세스를 구성하는 단계별 순서 관계
ProcessStep	NEXT_STEP	ProcessStep	프로세스 단계 간의 선후 실행 흐름 (BPMN Sequence Flow)
ProcessStep	EXECUTED_BY	Role	해당 단계를 직접 실행하는 실무 직무 역할
ProcessStep	USES_SYSTEM	System	해당 업무를 수행하기 위해 직원이 조작하는 소프트웨어 화면/모듈
ProcessStep	READS_TABLE	Table	업무 처리를 위해 읽기(SELECT) 권한으로 접근하는 테이블
ProcessStep	WRITES_TABLE	Table	업무 결과로 생성, 수정, 삭제(INSERT/UPDATE/DELETE)되는 테이블

이름 규칙 및 거버넌스 필수 속성

노드 라벨은 단수형 UpperCamelCase(예: TourProduct, ProcessStep)로 명명하며, 관계 타입은 방향성과 의미를 명시하는 동사형 대문자 UPPER_SNAKE_CASE(예: READS_TABLE, CALLS_API)로 고정한다. 속성 키는 소문자 lowerCamelCase(예: columnName, systemStatus)를 준수한다.

지식의 신뢰성과 계통 추적(Lineage)을 보장하기 위해, 모든 노드와 주요 관계에는 다음 6가지 거버넌스 메타데이터 속성을 필수(NOT NULL)로 부여한다.

- id: 시스템 전역 고유 식별자 (예: SYS_BOOKING_API, TBL_BOOKING_MASTER)
- name: 엔티티의 공식 명칭 (한글 및 영문 병기)
- owner: 해당 지식의 관리 책임 부서 또는 담당자 이메일
- source: 정보의 획득 출처 (예: tbls:mysql-booking-cluster, manual:ops-interview-v1)
- validator: 해당 지식의 비즈니스 유효성을 승인한 데이터 스튜어드 이메일
- updatedAt: 최종 변경 일시 (ISO 8601 타임스탬프)

재사용 국제 표준 어휘 체계

데이터의 고립을 방지하고 글로벌 상호운용성을 확보하기 위해 다음 6가지 표준 온톨로지를 사내 어휘에 투영한다.

- **W3C ORG**: Department와 Role 클래스는 org:OrganizationalUnit 및 org:Role과 일대일 대응되어 조직 개편 시에도 표준적인 상하 관계와 멤버십을 표현한다.
- **W3C PROV-O**: 지식의 출처 추적을 위해 source, validator, updatedAt 속성을 prov:wasDerivedFrom, prov:wasAttributedTo의 개념과 매핑한다.
- **W3C DCAT**: Database, Table, Column 메타데이터를 dcat:Catalog, dcat:Dataset, dcat:Distribution 체계로 정렬하여 데이터 카탈로그 표준을 준수한다.
- **W3C SKOS**: 비즈니스 용어사전 구축 시 표준어(skos:prefLabel)와 현업 동의어(skos:altLabel)를 매핑하여 "공급사"와 "파트너", "오더"와 "예약" 간의 개념 정합성을 확립한다.
- **schema.org**: TourProduct, Booking, Payment 등의 비즈니스 엔티티는 schema:TouristTrip, schema:Reservation, schema:Order 표준 어휘의 속성 구조를 벤치마킹한다.
- **BPMN 2.0** 온톨로지: BusinessProcess, ProcessStep의 흐름 구조는 BPMN의 Activity, SequenceFlow, Lane 개념 모델을 직접 원용한다.

실제 구현 스크립트 명세

데이터 적재 스크립트 예시 **1:** 시스템**,** 데이터베이스**,** 테이블 및 비즈니스 엔티티 연결

Cypher

```
MERGE (sys:System {id: "SYS_BOOKING_API"})
ON CREATE SET
  sys.name = "예약 API 서비스",
  sys.owner = "dev-backend@zoomzoomtour.com",
  sys.source = "system-catalog-2026",
  sys.validator = "architect@zoomzoomtour.com",
  sys.updatedAt = datetime().epochMillis;

MERGE (db:Database {id: "DB_BOOKING_MYSQL"})
ON CREATE SET
```

```
db.name = "MySQL Booking Cluster"
db.engine = "MySQL 8.0"
db.owner = "dba@zoomzoomtour.com"
db.source = "tbls-extractor"
db.updatedAt = datetime().epochMillis

MERGE (sys)-[:USES_DB]->(db)

MERGE (tbl:Table {id: "TBL_BOOKING_MASTER"})
ON CREATE SET
tbl.name = "bookings"
tbl.comment = "고객의 투어 상품 예약 원천 정보를 저장하는 핵심 마스터 테이블"
tbl.owner = "dev-backend@zoomzoomtour.com"
tbl.source = "tbls-extractor"
tbl.validator = "dba@zoomzoomtour.com"
tbl.updatedAt = datetime().epochMillis

MERGE (entity:Booking {id: "ENT_BOOKING"})
ON CREATE SET
entity.name = "예약 엔티티"
entity.description = "투어 상품 구매 및 이용 계약 엔티티"
entity.standardUri = "https://schema.org/Reservation"

MERGE (db)-[:HAS_TABLE]->(tbl)
MERGE (tbl)-[:REPRESENTS_ENTITY]->(entity);
```

데이터 적재 스크립트 예시 **2:** 외부 **PG** 연동 및 외래키 참조 관계 연결

Cypher

```
MERGE (ext:ExternalAPI {id: "EXT_TOSS_PAYMENTS"})
ON CREATE SET
ext.name = "토스페이먼츠 결제 승인 API"
ext.provider = "Toss"
ext.owner = "payment-dev@zoomzoomtour.com"
ext.source = "api-specs-v2"
ext.updatedAt = datetime().epochMillis

MATCH (sys:System {id: "SYS_BOOKING_API"})
MERGE (sys)-[:CALLS_API {protocol: "REST/HTTPS", timeoutMs: 3000}]->(ext);
```

```
MERGE (tblPay:Table {id: "TBL_PAYMENT_TRANSACTION"})
ON CREATE SET
  tblPay.name = "payment_transactions",
  tblPay.comment = "PG사 결제 승인/취소 트랜잭션 기록 테이블",
  tblPay.owner = "payment-dev@zoomzoomtour.com",
  tblPay.source = "tbls-extractor",
  tblPay.updatedAt = datetime().epochMillis;

MATCH (tblBook:Table {id: "TBL_BOOKING_MASTER"})
MERGE (tblPay)-[:FK_REFERENCES {fkColumn: "booking_id", pkColumn: "id"}]->(tblBook);
```

데이터 적재 스크립트 예시 **3:** 부서**-**업무 프로세스**-**실행 단계**-**테이블 **CRUD** 연결

Cypher

```
MERGE (dept:Department {id: "DEPT_OPERATIONS"})
ON CREATE SET dept.name = '운영팀', dept.standardUri =
'http://www.w3.org/ns/org#OrganizationalUnit';

MERGE (role:Role {id: "ROLE_TOUR_OPERATOR"})
ON CREATE SET role.name = "투어 배차 및 예약 관리자";

MERGE (dept)-[:HAS_ROLE]->(role);

MERGE (proc:BusinessProcess {id: "PROC_BOOKING_CONFIRMATION"})
ON CREATE SET
  proc.name = "예약 확정 및 바우처 발권 프로세스",
  proc.domain = "투어 운영";

MERGE (role)-[:RESPONSIBLE_FOR]->(proc);

MERGE (step1:ProcessStep {id: "STEP_CHECK_PARTNER_AVAILABILITY"})
ON CREATE SET step1.name = "1단계: 현지 공급사 예약 가능 여부 확인", step1.sequence =
1;

MERGE (step2:ProcessStep {id: "STEP_ISSUE_VOUCHER"})
ON CREATE SET step2.name = "2단계: 예약 확정 상태 변경 및 바우처 발급",
step2.sequence = 2;
```

```
MERGE (proc)-[:COMPOSED_OF]->(step1)
MERGE (proc)-[:COMPOSED_OF]->(step2)
MERGE (step1)-[:NEXT_STEP]->(step2)

MATCH (tblBook:Table {id: "TBL_BOOKING_MASTER"})
MATCH (sys:System {id: "SYS_BOOKING_API"})

MERGE (step1)-[:EXECUTED_BY]->(role)
MERGE (step1)-[:USES_SYSTEM]->(sys)
MERGE (step1)-[:READS_TABLE]->(tblBook)

MERGE (step2)-[:EXECUTED_BY]->(role)
MERGE (step2)-[:USES_SYSTEM]->(sys)
MERGE (step2)-[:WRITES_TABLE {action: "UPDATE_STATUS"}]->(tblBook)
```

질의 스크립트 예시 **1:** 스키마 변경 시 영향받는 시스템 및 부서 역추적 **(Impact Analysis)**

Cypher

```
MATCH (t:Table {name: "bookings"})
OPTIONAL MATCH (sys:System)-[:USES_DB]->(:Database)-[:HAS_TABLE]->(t)
OPTIONAL MATCH (step:ProcessStep)-[crud]->(t)
OPTIONAL MATCH (proc:BusinessProcess)-[:COMPOSED_OF]->(step)
OPTIONAL MATCH (role:Role)<-[:EXECUTED_BY]-(step)
OPTIONAL MATCH (dept:Department)-[:HAS_ROLE]->(role)
RETURN
  t.name AS TargetTable,
  collect(DISTINCT sys.name) AS ImpactedSystems,
  collect(DISTINCT proc.name) AS ImpactedProcesses,
  collect(DISTINCT step.name) AS ImpactedSteps,
  collect(DISTINCT dept.name) AS AlertDepartments
```

질의 스크립트 예시 **2:** 특정 부서가 의존하는 외부 연동 **API** 및 테이블 전수 식별

Cypher

```
MATCH (dept:Department {name: "물류부"})-[:HAS_ROLE]->(r:Role)
MATCH (r)<-[:EXECUTED_BY]-(step:ProcessStep)
```

```
OPTIONAL MATCH (step)-[:USES_SYSTEM]->(sys:System)-[:CALLS_API]->(api:ExternalAPI)
OPTIONAL MATCH (step)-[:READS_TABLE|WRITES_TABLE]->(tbl:Table)
RETURN
  dept.name AS Department,
  collect(DISTINCT sys.name) AS SystemsUsed,
  collect(DISTINCT api.name) AS ExternalAPIs,
  collect(DISTINCT tbl.name) AS DependentTables;
```

질의 스크립트 예시 **3:** 데이터 흐름 무결성 검증 **(**생성 주체 없이 조회만 되는 고립 테이블 탐색**)**

```
Cypher
MATCH (tbl:Table)
WHERE (0-[:READS_TABLE]->(tbl)) AND NOT (0-[:WRITES_TABLE]->(tbl))
RETURN
  tbl.id AS TableID,
  tbl.name AS TableName,
  tbl.owner AS Owner,
  '경고: 테이블을 생성하거나 갱신하는 업무 프로세스가 온톨로지에 정의되지 않았습니다.'
AS IssueDescription;
```

6. 단계별 구축 로드맵**:** 파일럿에서 전사 확장까지

단 1명의 개발 인력(주 1~2일 투입)으로 위험을 최소화하고 실행력을 확보하기 위해, 예약-확정-발권 프로세스 1건을 대상으로 4주 파일럿을 완수한 후 전사로 확산하는 10단계 로드맵을 적용한다.

단계	기간	활동 내용	주담당자	산출물	**AI** 에이전트 **(Claude Code)** 활용 방식	완료 판정 기준 **(DoD)**
1. 범위 정의	1주차	파일럿 대상 확정 (가장 복잡하고 파급력이	데이터 아키텍트, 운영팀 리드	파일럿 범위 정의서	기존 위키에서 예약 프로세스 관련 문서	대상 범위와 연관 3개 시스템(예약,

		큰 투어 예약-확정-발권 워크플로우 한정)			요약 및 범위 제안	어드민, PG) 확정
2. 개념 도출	2주차	도메인 핵심 엔티티 분류 (예약, 상품, 바우처, 파트너 등 10대 핵심 개념 정립)	데이터 아키텍트	핵심 엔티티 용어집	비즈니스 회의록 및 기안 문서를 분석하여 엔티티 후보군 자동 추출	표준 용어집 초안 승인 및 동의어 매핑 정의
3. 관계 정의	2주차	개념 간 카디널리티 및 상호 연결 규칙 수립 (BPMN 및 W3C 표준 어휘 정렬)	데이터 아키텍트	논리적 온톨로지 명세서	엔티티 간 관계 타입 및 제약 조건 자동 추천 프롬프트 가동	클래스-관계 간 메타모델 상호 검토 완료
4. 인프라 설치	3주차	사내 Ubuntu 서버에 Docker 기반 Neo4j CE 및 mcp-neo4j 컨테이너 가동[19]	개발자 (DevOps)	docker-compose.yml, 컨테이너 환경	Docker Compose 및 내부 네트워크 격리 스크립트 코드 생성	포트 바인딩 및 컨테이너 헬스체크 통과[11]
5. 스키마	3주차	원천	개발자	5개 DB	tbls.yml	600개

추출		DB(MySQL 4대, PG 1대)에 tbls를 실행하여 600개 테이블 메타데이터 JSON 추출[2]		schema.json 및 마크다운 문서[2]	설정 생성 및 추출 JSON 유효성 검증 스크립트 작성[26]	테이블 메타데이터 완전 파싱 확인
6. 프로세스 모델링	4주차	운영팀·CS팀 인터뷰를 통해 예약-발권 프로세스 단계별 역할 및 테이블 CRUD 명문화	아키텍트, 부서 스튜어드	프로세스-데이터 CRUD 매트릭스	인터뷰 녹취 요약본을 BPMN 정형 텍스트 및 JSON 구조로 자동 변환	운영팀 리드의 단계별 테이블 매핑 사실 확인 서명
7. 데이터 적재	4주차	schema.json과 모델링 데이터를 Cypher 스크립트로 변환하여 Neo4j 초기 적재[27]	개발자	초기 파일럿 지식그래프	JSON to Cypher MERGE 변환 파이썬 스크립트 생성 및 디버깅	노드 및 관계 적재 오류 0건 및 인덱스 생성[4]
8. 에이전트 연결	5주차	mcp-neo4j-cypher 서버를 Claude Desktop/Code	개발자, 파일럿 테스터	MCP 클라이언트 설정 가이드	클라이언트별 claude_desktop_config.json 설정 파일	자연어 질의를 통한 예약 테이블 영향도

		클라이언트에 연결[32]			자동 생성[33]	정확 조회
9. 협업·승인 절차	6주차	지식 수정 제안을 위한 Git PR 템플릿 및 자동 머지 파이프라인(GitOps) 구축	개발자, 아키텍트	GitOps 거버넌스 파이프라인	자연어 제안을 분석하여 Cypher 패치 스크립트 자동 생성봇 구성	테스트 제안 건의 PR 발행-승인-적재 사이클 완주
10. 유지보수 및 전사 확장	7~16주차	주 단위로 정산, 상품기획, 마케팅 부서 순차 모델링 및 스키마 주기적 갱신	전 부서 스튜어드, 개발자	전사 지식그래프 완성본, 모니터링 체계	정기 tbls diff 결과를 분석하여 변경 영향도 보고서 자동 작성[2]	20대 역량 질문 정답률 95% 이상 달성

7. 협업 거버넌스 및 지식 최신성 유지 체계

팀원 제안-검토-반영 워크플로우

비전문가 팀원들이 지식그래프를 지속적으로 최신화할 수 있도록 AI 에이전트와 GitOps를 결합한 무마찰 제안 프로세스를 가동한다.

1. 자연어 기반 수정 제안: 팀원이 Claude Desktop 또는 Claude Code와의 대화 중 불일치를 발견할 경우(예: "정산 프로세스 3단계에서 읽는 테이블이 settlement_v2로 변경되었는데 그래프에는 과거 테이블로 표기되어 있어") 자연어로 수정을 제안한다. 팀원이 지식그래프 수정을 요청하면, AI 에이전트는 사전 정의된 시스템 프롬프트를 바탕으로 정형 변경 명세 YAML 파일을 자동 생성한다.
2. **Git Pull Request** 자동 생성: AI 에이전트는 GitHub MCP 도구를 활용하여 사내 지식 저장소 레포지토리에 이슈를 등록하거나, patch/settlement-process-table 브랜치를 생성하여 변경 YAML을 커밋하고 PR(Pull Request)을 오픈한다[34].
3. 데이터 스튜어드 검토 및 승인: 해당 비즈니스 도메인의 책임 부서 스튜어드(정산팀 담당자)와 데이터 아키텍트에게 알림이 전송된다. 스튜어드는 Git 인터페이스에서 비즈니스 변경 사항의 타당성을 검토하고 승인(Approve)을 수행한다.
4. **CI/CD** 파이프라인 자동 적재: PR이 main 브랜치에 병합되면 GitHub Actions 러너가 트리거된다. 러너는 사내 서버의 배치 스크립트를 호출하여 변경 사항을 Cypher MERGE

문으로 컴파일하고 Neo4j 인스턴스에 안전하게 반영한 후 처리 결과를 사내 협업 도구로 알린다.

부서별 데이터 스튜어드 역할 정의

지식의 소유권(Ownership)을 명확히 하고 거버넌스를 정착시키기 위해 부서별로 1인의 데이터 스튜어드를 공식 지정한다.

- 상품기획 스튜어드: 상품 카테고리 분류 체계, 공급사 계약 유형 관련 엔티티의 최신성 검증
- 운영/CS 스튜어드: 예약 접수, 배차, 확정, 취소/환불, 고객 클레임 처리 단계 및 시스템 매핑 검증
- 정산 스튜어드: 정산 산출 로직, 지급 명세 테이블 및 외부 은행 연동 관계 승인
- 개발/DBA 스튜어드: 물리 스키마 변경, 외래키 정의, 시스템 모듈 간 API 연동 최신화 책임

지식 노후화 방지 메커니즘

지식그래프가 시간이 지남에 따라 실제 시스템과 괴리되는 현상(Knowledge Rot)을 방지하기 위해 2단계 동기화 체계를 가동한다.

- 스키마 수준 자동 감지: 매일 새벽 tbls diff 명령이 CI 파이프라인에서 실행된다[2]. 실제 DB 스키마와 온톨로지 메타데이터 간의 차이가 감지되면 즉시 GitHub Issue를 생성하고 DBA 스튜어드에게 할당한다.
- 분기별 프로세스 리프레시 워크숍: 각 부서 스튜어드와 함께 분기당 1회 지식그래프 검증 세션을 진행하여, 최근 개정된 사내 업무 매뉴얼과 온톨로지 간의 간극을 일괄 점검하고 패치한다.

8. 엔터프라이즈 보안 및 AI 접근 제어 아키텍처

원천 DB 접근 계정 격리

메타데이터를 수집하는 tbls 전용 데이터베이스 사용자 계정을 원천 DB별로 독립 생성한다[2]. 이 계정은 고객 개인정보나 결제 정보 등 실제 비즈니스 데이터 본문(SELECT on data tables)에 접근할 수 없도록 권한을 박탈한다. 오직 데이터 딕셔너리 및 시스템 카탈로그 메타데이터만 열람할 수 있도록 권한을 최소화한다.

- **MySQL**: GRANT SELECT ON information_schema.* TO 'tbls_agent'@'%';
- **PostgreSQL**: GRANT CONNECT ON DATABASE zoomzoom TO tbls_agent; GRANT USAGE ON SCHEMA public TO tbls_agent; (일반 테이블에 대한 SELECT 권한은 일절 부여하지 않음)

MCP 서버 네트워크 노출 통제

사내 서버의 Docker 환경에서 실행되는 mcp-neo4j-cypher 컨테이너는 공인 IP에 노출되지 않도록 엄격히 통제한다[11]. 사무실 내부 서브넷 및 사내 VPN(WireGuard 또는 OpenVPN)을 통해서만 통신할 수 있도록 방화벽을 구성한다.

또한 MCP 실행 환경변수에서 읽기 전용 플래그(--read-only)를 설정하고, Cypher 파서 레벨에서 데이터 조작 언어(DML) 및 데이터 정의 언어(DDL) 키워드가 인입될 경우 즉시 차단하는 샌드박싱 정책을 적용한다[4].

클라우드 LLM 전송 위험 분석 및 로컬 LLM 대안

팀원들이 Claude Code, Claude Desktop, Cursor 등 상용 클라우드 AI 에이전트를 활용하므로 사내 구조 정보 전송에 따른 위험도를 평가하고 대안을 마련한다.

온톨로지 지식그래프에는 고객 이름, 주민등록번호, 연락처, 카드번호 등 개인식별정보(PII)와 금융 거래 원천 데이터가 일절 포함되지 않는다. 따라서 개인정보보호법상 실데이터 유출 리스크는 원천 차단된다.
그러나 시스템 모듈 간 연동도, DB 테이블 및 컬럼 구조, 사내 비즈니스 프로세스 흐름 자체는 기업의 영업 비밀에 해당할 수 있다. Anthropic의 기업용 플랜(Claude Team 또는 Enterprise)은 고객이 입력한 프롬프트와 컨텍스트 데이터를 모델 학습에 활용하지 않는 비학습(Zero Data Retention/Non-training) 정책을 보장하므로 사내 에이전트 도입 시 비즈니스 플랜 사용을 의무화해야 한다.
만약 회사 보안 규정상 시스템 메타데이터조차 외부 클라우드로 전송되는 것을 금지할 경우, 사내 단일 서버(32GB RAM) 내에 완전 격리형 로컬 LLM 환경을 구축할 수 있다. 사내 서버에 Ollama 또는 vLLM 컨테이너를 가동하고, 4비트 양자화된 Qwen2.5-Coder-7B-Instruct 또는 Llama-3.1-8B-Instruct 모델(약 5GB~6GB VRAM/RAM 점유)을 서빙한다.
팀원들의 에이전트 클라이언트가 외부 Anthropic 엔드포인트 대신 사내 로컬 LLM 엔드포인트(http://internal-server:11434/v1)를 호출하도록 프록시를 설정함으로써 100% 사내 에어갭(Air-gapped) 환경을 완성할 수 있다.

9. 지식그래프 검증을 위한 **20**대 역량 질문 및 평가 체계

구축된 온톨로지와 지식그래프가 실질적인 비즈니스 및 엔지니어링 문제를 해결할 수 있는지 검증하기 위해 5개 도메인에 걸쳐 20개의 역량 질문(Competency Questions, CQ)을 도출한다.

시스템 아키텍처 및 연동 도메인

1. 예약 *API* 서비스*(SYS_BOOKING_API)*가 직접 연동하는 모든 외부 *PG* 및 공급사 *API* 목록은 무엇인가*?*
2. 토스페이먼츠 승인 *API* 장애 발생 시 직접적인 영향을 받는 사내 시스템 모듈은 무엇인가*?*
3. *MySQL Booking* 인스턴스에 물리 커넥션을 맺고 있는 모든 애플리케이션 서비스 목록은*?*
4. 현지 공급사 투어 연동 *API*를 호출하는 백엔드 모듈과 해당 시스템의 담당 개발자는 누구인가*?*

DB 스키마 및 데이터 거버넌스 도메인

5. *'bookings'* 마스터 테이블을 참조하는 모든 외래키*(FK)*를 가진 하위 테이블 목록은*?*
6. *'settlement_transactions'* 테이블의 비즈니스 소유 부서와 스키마 검증자는 누구인가*?*
7. 시스템 전체에서 고객 환불 상태*(refund_status)* 데이터를 관리하는 모든 테이블과 컬럼은 무엇인가*?*
8. *MySQL 4*대와 *PostgreSQL 1*대를 통틀어 *'partner_id'*를 컬럼으로 보유한 테이블의 전수 목록은*?*

영향도 분석 **(Impact Analysis)** 도메인

9. *'bookings'* 테이블의 특정 컬럼 타입을 변경할 때 영향을 받는 부서와 세부 업무 단계는*?*
10. 정산 시스템 *DB* 마이그레이션 시 사전에 작업을 공유해야 하는 모든 유관 부서 이해관계자는*?*
11. 특정 레거시 테이블을 *DROP*할 경우 어떤 프로세스 단계에서 런타임 오류가 발생하는가*?*
12. 외부 *PG* 모듈의 타임아웃 설정을 축소할 때 연관된 비즈니스 프로세스 단계는 무엇인가*?*

비즈니스 워크플로우 및 조직 프로세스 도메인

13. '예약 확정 및 바우처 발권' 프로세스의 *2*단계에서 읽고 쓰는 구체적인 테이블 목록은*?*
14. 운영팀의 '투어 배차 관리자' 역할이 일상 업무에서 조작하는 모든 사내 시스템 화면은*?*

15. 투어 상품 취소 시 실행되는 환불 프로세스의 전체 세부 단계와 선후 실행 순서는?
16. 정산 부서가 파트너 대금을 송금하기 전에 반드시 완료되어야 하는 선행 업무 단계는?

고객 지원 및 운영 거버넌스 도메인

17. 고객이 '바우처 미수신'으로 *CS* 문의를 접수했을 때 *CS* 담당자가 조회해야 하는 시스템과 테이블은?
18. 비즈니스 엔티티 '바우처'의 생성부터 소멸까지 관여하는 모든 업무 단계와 시스템 흐름은?
19. 신규 입사한 운영팀 직원이 담당 역할을 수행하기 위해 숙지해야 하는 업무 프로세스 맵은?
20. 최근 *30*일 이내에 스키마나 프로세스 정의가 변경되었으나 아직 스튜어드의 최종 승인을 받지 못한 엔티티는?

역량 질문 평가 및 측정 방법론

정량 평가는 정기 배포 시마다 위 20개 질문을 자연어로 Claude 에이전트에 입력하여 수행한다.

- 쿼리 생성 성공률**(Cypher Generation Accuracy)**: 에이전트가 문법 오류 없이 올바른 Cypher를 생성하여 실행한 비율을 측정하며, 목표치는 95% 이상으로 설정한다.
- 정보 정합성**(Recall/Precision)**: 실제 아키텍처 및 ERD 사실 대비 누락되거나 왜곡된 결과가 없는지 아키텍트가 교차 검증하며, 목표치는 100%이다.
- 경로 탐색 깊이**(Hop Coverage)**: 부서 -> 역할 -> 단계 -> 시스템 -> DB -> 테이블로 이어지는 3홉 이상의 복합 다차원 그래프 순회가 정확히 완수되는지 점검한다.

10. 실패 요인 분석**,** 예방책 및 단계별 확장 판단 기준

흔한 실패 요인과 선제적 예방책

과도한 온톨로지 엄밀성 추구로 인한 조기 탈진(Over-Engineering Trap)을 경계해야 한다. 시맨틱 웹의 학술적 정의에 매몰되어 복잡한 OWL 공리와 추론 규칙 설계에 집착할 경우, 실무 데이터 적재 전에 프로젝트가 동력을 상실하기 쉽다. 따라서 복잡한 RDF/OWL 도구 대신 직관적인 속성 그래프(LPG) 모델을 채택하고 실무에 필수적인 최소한의 라벨과 관계로 출발해야 한다.

수동 데이터 입력 피로도(Manual Entry Fatigue) 역시 주된 실패 요인이다. 600개 테이블과 업무 매뉴얼을 개발자가 일일이 수작업으로 그래프에 입력할 경우 작업 지연과 오류를 피할 수 없다. tbls를 통해 600개 테이블 메타데이터를 자동 수집하고[2], 프로세스 정의도 인터뷰 문서를 Claude Code에 입력하여 JSON 구조로 변환하는 반자동화 파이프라인을 구축해야 한다.

거버넌스 부재로 인한 지식의 급속한 노후화(Knowledge Rot)는 지식그래프의 신뢰도를 무너뜨린다. 구축 이후 DB 스키마가 계속 변경됨에도 그래프가 동기화되지 않으면 팀원들은 시스템을 외면하게 된다. 이를 예방하기 위해 CI 파이프라인에 tbls diff를 연동하여 스키마 변경 시 자동 경고를 발송하고[2], 부서별 데이터 스튜어드 승인제를 운영 수칙으로 정착시킨다.

AI 에이전트의 환각(Hallucination) 및 오류 쿼리 생성 문제도 사전 차단해야 한다. LLM이 그래프 스키마 구조를 왜곡하여 존재하지 않는 라벨이나 관계를 조회하는 것을 방지하기 위해, MCP 구동 시 get_neo4j_schema 도구를 통해 그래프 스키마 컨텍스트를 에이전트 시스템 프롬프트에 상시 주입해야 한다[32].

단계별 확장 판단 기준

구축 착수(M0) 후 4주간의 1개 프로세스 대상 파일럿(M1)을 거쳐 전사로 확산하는 과정에서 단계별 통과 기준을 엄격히 적용한다.



1. 파일럿 완료 시점 **(M1, 4**주차**)**: 예약-발권 프로세스 1건에 대한 스키마 및 프로세스 매핑 완료 여부, 파일럿 테스터 3명의 역량 질문 5건 정확 응답 달성 여부를 확인하고 다음 단계로 진입한다.
2. **3**개월 시점 평가 **(**중간 확장 단계**)**:
   - 정량 지표: 핵심 시스템 3개 이상(예약, 어드민, 결제) 및 200개 이상 테이블 적재 완료, 역량 질문 20개 중 10개에 대해 AI 에이전트 질의 성공률 90% 이상 달성, 대상 팀원 15명 중 주 1회 이상 지식그래프를 조회하는 주간 활성 사용자(WAU) 비율 50%(8명) 이상.
   - 판단 및 조치: 기준 충족 시 정산, 마케팅, CS 부서 및 잔여 테이블 전체 모델링에 착수한다. 기준 미달 시 사용자 인터뷰를 통해 프롬프트 템플릿과 노드 속성 표현을 개선하는 튜닝 스프린트를 우선 수행한다.
3. **6**개월 시점 평가 **(**전사 안착 단계**)**:
   - 정량 지표: 전사 10개 시스템, 600개 전체 테이블, 주요 10대 핵심 비즈니스 프로세스 100% 매핑 완료, 역량 질문 20개 전체에 대한 정답률 95% 이상 달성, 스키마 동기화 지연시간 24시간 이내 유지, Git PR을 통한 자발적 지식 수정 제안 월 3건 이상 발생.
   - 판단 및 조치: 지식그래프가 사내 단일 진실 공급원(Single Source of Truth)으로 정착된 것으로 평가하고, 신규 입사자 온보딩 및 시스템 개편 영향도 평가의 공식 필수 절차로 지정한다. 만약 질의 트래픽과 동시 요청이 급증하여 단일 서버 리소스 한계에 도달할 경우, 고가용성 클러스터 지원 솔루션(Neo4j Enterprise 또는 분산 카탈로그)으로의 인프라 전환 타당성을 검토한다.

11. 도입 전 직접 확인해야 할 체크리스트

실제 시스템 설치 및 온톨로지 구축에 착수하기 전, 엔지니어링 및 거버넌스 측면에서 점검해야 할 항목이다.

1. 인프라 및 네트워크 환경

- [ ] 사내 Ubuntu 서버(8 vCPU/32GB)의 루트 디스크 가용 용량이 최소 50GB 이상 확보되어 있는가?
- [ ] Docker Engine 24+ 및 Docker Compose v2가 정상 설치되어 비루트(non-root) 실행이 가능한가[41]?
- [ ] 사내 방화벽 및 보안 그룹 설정에서 외부 공인 인터넷으로부터의 7474, 7687 포트 접근이 원천 차단되어 있는가[11]?
- [ ] 서버의 NTP 시간 동기화 서비스가 정상 동작하여 감사 타임스탬프의 오차가 발생하지 않는가[11]?

2. 원천 데이터베이스 권한 체계

- [ ] 운영 중인 MySQL 4대 및 PostgreSQL 1대에 각각 tbls_agent 전용 계정이 생성되었는가[2]?
- [ ] 해당 계정들이 비즈니스 실데이터(SELECT on tables)에는 접근할 수 없고 오직 information_schema 및 시스템 메타데이터 카탈로그만 읽을 수 있도록 권한이 최소화되었는가?
- [ ] 사내 서버 IP로부터 각 원천 DB 서버(포트 3306, 5432)로의 내부망 TCP 인바운드 접근이 방화벽상에서 개방되어 있는가?

3. 라이선스 및 컴플라이언스

- [ ] 사내 법무/보안 정책상 Neo4j Community Edition(GPL v3)을 소스코드 수정 없이 내부 업무용 컨테이너로 구동하는 데 이견이 없는지 확인하였는가[13]?
- [ ] 팀원들이 사용하는 Claude 계정이 입력 컨텍스트를 모델 학습에 활용하지 않는 비즈니스(Team 또는 Enterprise) 플랜인지 확인하였는가?
- [ ] 지식그래프 적재 데이터 정의서에서 고객 개인정보(이름, 연락처, 이메일, 주민번호) 및 결제 금융 정보가 원천 배제되었는가?

4. 개발 및 운영 거버넌스

- [ ] 전담 엔지니어 1명의 주 1~2일 작업 공수가 공식 스프린트 업무로 배정되었는가?
- [ ] 지식 버전 관리를 위한 사내 Git 저장소(zoomzoom-knowledge-graph)가 생성되었는가?
- [ ] 상품기획, 운영, CS, 정산, 마케팅 부서에서 최소 1명의 협업 담당자(데이터 스튜어드)가 공식 지정되었는가?
- [ ] 보안 취약점(CVE-2026-35402)이 해결된 최신 MCP 서버 이미지(mcp-neo4j-cypher >= v0.6.0)를 사용하도록 구성되었는가[33]?

참고 자료

1. Introduction - Operations Manual - Neo4j, https://neo4j.com/docs/operations-manual/current/introduction/
2. k1LoW/tbls: database docs and ER diagrams in Markdown, https://hysenlabs.com/projects/k1low-tbls
3. OpenMetadata - Modern DataTools, https://www.modern-datatools.com/tools/openmetadata
4. Thinking in Graphs: Neo4j-Backed Exploit-Chain Reasoning for LLM,

https://arxiv.org/html/2610.20902v1
5. GraphDB Documentation - Ontotext, https://graphdb.ontotext.com/documentation/11.1/pdf/GraphDB.pdf
6. Enterprise Knowledge Graph Platforms Compared 2026 - Future AGI, https://futureagi.com/blog/enterprise-knowledge-graph-platforms-2026/
7. Best Graph Databases in 2026: A Buyer's Guide - TigerGraph, https://www.tigergraph.com/blog/best-graph-databases/
8. conceptkernel/jena-fuseki - Docker Image, https://hub.docker.com/r/conceptkernel/jena-fuseki
9. Apache Jena Fusekiとは？Jena 6の変更点とDocker導入手順【2026, https://www.issoh.co.jp/tech/details/10916/
10. The webprotege code base with an export functionality to a Neo4j, https://github.com/woschmid/webprotege-export-neo4j
11. Hardened Neo4j 5.26 on CentOS Stream 9 | AWS Marketplace, https://aws.amazon.com/marketplace/pp/prodview-dqan7galdieou
12. Context Engineering Tools for AI Agents: Complete 2026 Guide - Atlan, https://atlan.com/know/context-engineering/context-engineering-tools-for-ai-agents/
13. r/Neo4j on Reddit: Just read this comment that scared me. What are, https://www.reddit.com/r/Neo4j/comments/jpmkth/just_read_this_comment_that_scared_me_what_are/
14. neo4j: backup and restore data - Stack Overflow, https://stackoverflow.com/questions/76226546/neo4j-backup-and-restore-data
15. Dump and load a Neo4j database (offline) - Operations Manual, https://neo4j.com/docs/operations-manual/current/docker/dump-load/
16. Deploy on GCP - PipesHub Workplace AI - Introduction, https://pipeshub.mintlify.app/deployment/vendor/gcp
17. Memgraph vs Neo4j: Graph Database Comparison - PuppyGraph, https://www.puppygraph.com/blog/memgraph-vs-neo4j
18. GitHub - knowall-ai/mcp-reverie: Memory management MCP server, https://github.com/knowall-ai/mcp-reverie
19. Installation - Operations Manual - Neo4j, https://neo4j.com/docs/operations-manual/current/installation/
20. Best Graph Databases for AI Agents - Integrate.io, https://www.integrate.io/blog/best-graph-databases-for-ai-agents/
21. memgraph/tests/mgbench/benchmark_context.py at master - GitHub, https://github.com/memgraph/memgraph/blob/master/tests/mgbench/benchmark_context.py
22. 10 Best Graph Databases for Voice AI Agents in 2026 - HydraDB, https://hydradb.com/blog/graph-databases-voice-ai-agents
23. Jena Fuseki docker image for DraCor - GitHub, https://github.com/dracor-org/dracor-fuseki
24. Publish a docker image up-to-date using a github actions workflow, https://github.com/apache/jena/issues/3165
25. Knowledge Graph Tools Compared: Features, Pricing, and Use Cases,

https://atlan.com/know/ai-agent/knowledge-graph/knowledge-graph-tools-compared/
26. k1LoW/tbls: tbls is a CI-Friendly tool to document a database, written, https://github.com/k1Low/tbls
27. Output schema data file by default · Issue #387 · k1LoW/tbls - GitHub, https://github.com/k1LoW/tbls/issues/387
28. The webprotege code base - GitHub, https://github.com/protegeproject/webprotege
29. webprotege-deploy - Compose file for WebProtégé services - GitHub, https://github.com/protegeproject/webprotege-deploy
30. soda-collections-objects-data-literacy/scs-webprotege: The ... - GitHub, https://github.com/soda-collections-objects-data-literacy/scs-webprotege
31. tbls - Liam ERD, https://liambx.com/glossary/tbls
32. neo4j-contrib/mcp-neo4j: Neo4j Labs Model Context Protocol servers, https://github.com/neo4j-contrib/mcp-neo4j
33. mcp-neo4j-cypher - PyPI, https://pypi.org/project/mcp-neo4j-cypher/
34. GitHub - AI-App/ModelContextProtocol.Servers: Model Context, https://github.com/AI-App/ModelContextProtocol.Servers
35. A unified Neo4j MCP server for GraphRAG: vector search ... - GitHub, https://github.com/neo4j-field/mcp-neo4j-graphrag
36. DrSnowbird/docker-webprotege: Docker image for WebProtégé, https://github.com/DrSnowbird/docker-webprotege
37. tbls Insights, https://insights.linuxfoundation.org/project/k1low-tbls
38. k1low/tbls - Docker Image, https://hub.docker.com/r/k1low/tbls
39. MCP Plugins for OpenCode - Extend with 1200+ Servers, https://opencode-tutorial.com/en/plugins
40. NVD-CVE-2026-35402 - NIST, https://nvd.nist.gov/vuln/detail/cve-2026-35402
41. OmniBioAI — AI Bioinformatics Platform, https://omnibioai.org/